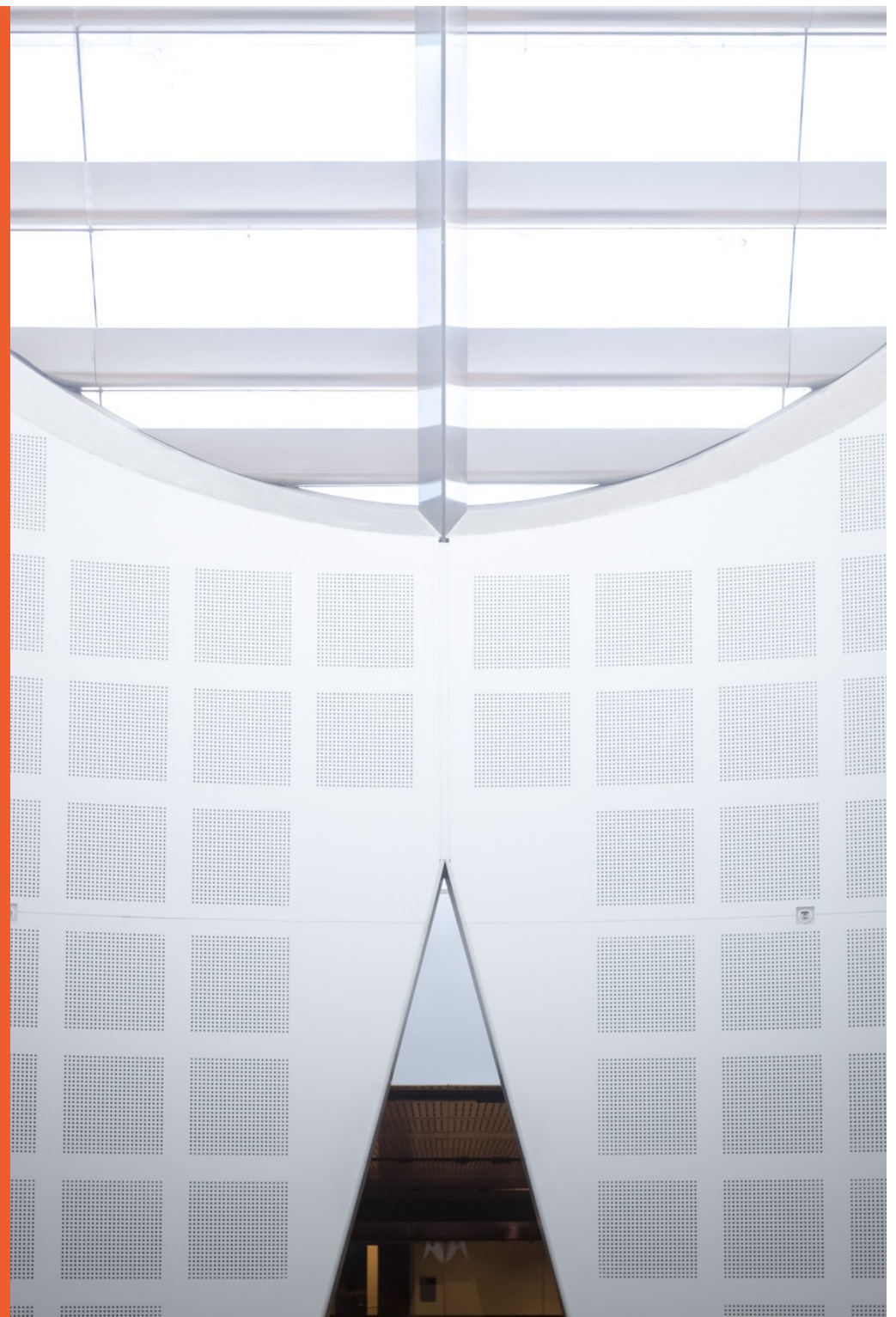


Client Server Communication & Backend Web Development

Presented by
Dr. Huaming Chen

ELEC3609/9609
Internet Software Platforms
2026 S2



We acknowledge the tradition of
custodianship and law of the Country on which
the University of Sydney campuses stand.
We pay our respects to those who have cared
and continue to care for Country.



THE UNIVERSITY OF
SYDNEY

Overview for client-server communication

- ✓ **XML/JSON**
- ✓ **HTTP**
 - ✓ **Requests**
 - ✓ **Cookies**
 - ✓ **Content Type**
 - ✓ **Referer**
 - ✓ **User-Agent**
- ✓ **Backend web development**

Client Server Communication

- Need a format and way to transmit/receive data.
- Some general client-server communication mechanisms
 - Sockets
 - Remote procedure calls
 - Pipes
- MIME (Multipurpose Internet Mail Extensions) type determines the file format and format contents
 - IANA
- Common examples:
 - text/html
 - image/png
 - application/json
 - text/css

Human vs. Machine



Huaming
Huaming-Chen

[Edit profile](#)

1 follower · 1 following

📍 Sydney, NSW, Australia

✉ huaming.chen@ieee.org

🔗 <https://www.sydney.edu.au/engineering/about/our-people/academic-staff/huaming-chen.html>

Organizations



Popula

advn

Forke

Adver

FairC

Repos

129

1 contr

Mon

Wed

Fri

Contributors

Septen

Seeing s

[illegible]

XML

- Extensible Markup Language (XML) defines a set of rules for encoding documents.
- It is a language for describing the rules of other languages. XML allows us to define schemas for tag-based languages. A schema is a contract for communication between client and server.
- XML is:
 - a markup language, *not* a programming language.
 - human-readable.
 - machine-readable.
 - designed to be self-descriptive.
 - designed to carry and transport data, not to display data.

XML Example

```
<?xml version="1.0"?>
<book language="en" author="Ernest Hemingway">
  <title>A Farewell to Arms</title>
  <chapter>
    <heading>Chapter One</heading>
    <paragraph> ... <paragraph>
    <paragraph>
  </chapter>
  <chapter>
    <heading>
    <paragraph>
    <paragraph>
  </chapter>
</book>
```

```
POST /InStock HTTP/1.1
Host: www.example.org
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 299
SOAPAction: "http://www.w3.org/2003/05/soap-envelope"

<?xml version="1.0"?>
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope"
xmlns:m="http://www.example.org">
  <soap:Header>
  </soap:Header>
  <soap:Body>
    <m:GetStockPrice>
      <m:StockName>T</m:StockName>
    </m:GetStockPrice>
  </soap:Body>
</soap:Envelope>
```

XML Terminology

- **Markup:** Elements that define the structure of the document.
Usually enclosed by:
 - `<` and `>`
 - `&` and `;`
- **Content:** All the information between *markup*.
- **Tag:** A type of *markup* that begin with `<` and ends with `>`, and comes in three flavours:
 - *start tag*: `<section>`
 - *end tag*: `</section>`
 - *empty-element tag*: ``
- **Attribute:** *Markup* on a *start tag* or *empty-element tag* that defines key-value pairs about that element.
 - ``

XML Document Type

- To define what types of fields are allowed in an XML document, and in what structure, a document type declaration (DOCTYPE) is provided.
- The DOCTYPE identifies a document type definition (DTD), which defines the fields and their structure.
- For our XML book document example, the DTD is:

```
<!ELEMENT book (title , chapter+)>
  <!ELEMENT chapter (heading , paragraph*)>
    <!ELEMENT heading (#PCDATA)>
    <!ELEMENT paragraph (#PCDATA)>
  <!ATTLIST book
    language CDATA #REQUIRED
    author CDATA #REQUIRED
  >
```

JSON

- JavaScript Object Notation (JSON) is an open-standard format for representing data online. As the name suggests, it is based on the representation of objects in JavaScript.
- It is **language-independent** and supported by most programming languages.
- Typically, it is used for:
 - API Requests/Responses
 - Serialization
 - Configuration files

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "city": "Paris",
  "temperature": 18,
  "condition": "Cloudy"
}
```

JSON Example

```
{
  "firstName": "John",
  "lastName": "Smith",
  "isAlive": true,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "mobile",
      "number": "123 456-7890"
    }
  ],
  "age": 25
}
```

HTTP

- Hypertext Transfer Protocol (HTTP) is designed to allow communication between a HTTP client and HTTP server.
- HTTP works as a request-response protocol. A client makes a request and the server responds, generally with a web page.

HTTP Request

- Clients sends a HTTP request to the server.
- Example request to www.elec3609.com/index.html:

```
GET /index.html HTTP/1.1  
Host: www.elec3609.com
```

HTTP Response

- The server sends a HTTP response:

```
HTTP/1.1 200 OK
Server: nginx/1.10.0 (Ubuntu)
Date: Fri, 26 Aug 2016 03:55:14 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Set-Cookie: __session_x=KGRwMApTJ3VzZXJfaWQnAxCkg.CqFRyw.1
           hMMxByGE9m4iH7ssE5guYI; HttpOnly; Path=/
Content-Encoding: gzip

<HTML> ... </HTML>
```

- This response is then rendered by a browser to display a web page.

HTTP request example

Now that we have an open TCP connection to the web server we just need to send it a HTTP request through that connection.

A HTTP Request

```
GET /students/view-unit-page/alpha/ELEC3609 HTTP/1.1
Host: cusp.sydney.edu.au
Connection: keep-alive
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/49.0.2623.87 Safari/537.36
Accept-Encoding: gzip, deflate, sdch
Accept-Language: en-AU,en;q=0.8,en-GB;q=0.6
Cookie: km_ai=IaRyuapbUDqQ%3D; km_lv=x; km_uq=;
        BIGipServer~APPLICATION~cusp.sydney.edu.au_443=
        rd1o000000000000000000000000ffffac140537o80; _jsuid
```

HTTP status code

- The response can specify a HTTP status code which indicate the status

HTTP code category	Definition	Example
1xx	Informational	100: Continue 101: Switching Protocols
2xx	Success	200: OK – the standard response
3xx	Redirection	301: Moved Permanently 302: Found/Moved Temporarily (similar to 303 & 307) 304: Not Modified
4xx	Client	404: Not found
5xx	Server	502: Bad gateway

HTTP Structure

- A HTTP request or responses consists of:
 - **Header:** Contain meta-data such as cookie and connection information.
 - **Body:** Contains the data to be sent with the transmission (optional).
- There are several types of web requests:
 - **GET:** A simple request header only
 - **POST:** A request containing data in the body, e.g. form data, file upload, etc.
 - **HEAD:** The same as GET, except server omits the body in the response (only sends header)
 - **PUT:** Replace a target resource on the server
 - **PATCH:** Partially modify
 - **DELETE:** Delete a target resource on the server
 - **OPTIONS:** Ask the server which request types are allowed against a particular resource

HTTP Content Type

- Generally, a HTTP request uses the *Accept* header to indicate to the server which data types it will accept in the response. A client can define multiple data types that it supports.
- A HTTP response defines the *Content-Type* header, which determines what type of data is contained in the body of the response. This is an indication to the browser as to how it should interpret the response.
- *Content-Type* applies to both request and response bodies.

```
GET /data HTTP/1.1  
Accept: application/json
```

```
HTTP/1.1 200 OK  
Content-Type: application/json  
  
{ "temp": 18, "city": "Paris" }
```

HTTP Cookies

- A HTTP cookie is a small piece of data, set by a website, which is stored in the user's browser.
- Browsers automatically include all the cookies set by a server with every HTTP request sent to that server.
- There are two HTTP headers which control cookies on the web:
 - **Requests:** Cookie
 - Cookie: username=abc123; authToken=4f3e
 - **Responses:** Set-Cookie
 - Set-Cookie: username=abc123
 - Set-Cookie: authToken=4f3e

HTTP Cookies II

- Cookies are often used for tracking users and authentication.
- A typical login flow:
 - 1. User visits website for first time.
 - no cookie
 - 2. User submits login form to server with username + password.
 - still no cookie
 - 3. Server generates authentication token (auth-tok).
 - Server responds with Set-Cookie: token=[auth-tok]
 - 4. User visits website.
 - Client sends: Cookie: token=[auth-tok]
 - 5. Server recognises (auth-tok) and knows request came from User.

HttpOnly, Secure, SameSite

HTTP Referer

- The HTTP *Referer* header is sent with requests made by a browser. It indicates to the server, the previous page that the user was on.
- For example:
 - I go to www.google.com and search for “elec3609”.
 - I click on the link to the www.elec3609.com website.
 - The browser’s request to www.elec3609.com includes a **Referer** header, containing our previous address:

<https://www.google.com/>

Referer → Referrer

‘Referer’ is a misspelling of the word **referrer**.

It was originally misspelled in [RFC 1945](#) due to neither spelling being present in the standard [UNIX spellchecker](#) of the time, and has remained that way ever since.

HTTP User-Agent

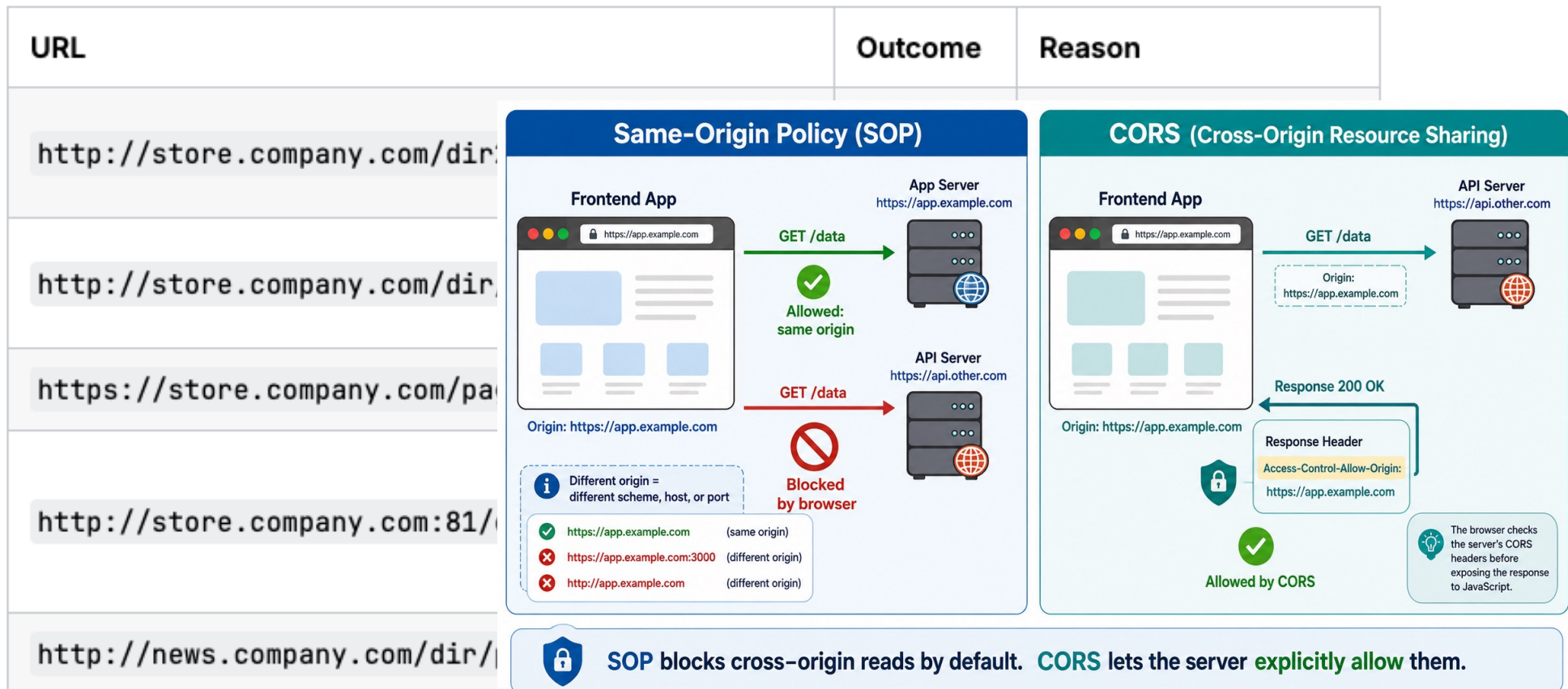
- The HTTP User-Agent header tells the server what type of software the client is using.
- Typically, it includes information like:
 - Browser
 - Rendering Engine
 - Operating System

Example: Chrome on Linux

```
Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,  
like Gecko) Chrome/51.0.2704.106 Safari/537.36
```

What happen when frontend and backend run on different origins/ports

- Two URLs have the same origin if the **protocol**, **port** (if specified), and **host** are the same for both.
- *http://store.company.com/dir/page.html*



Overview for backend web development

- ✓ **Data Modelling**
 - ✓ TIL
 - ✓ ERD
 - ✓ Model View Template/Controller
- ✓ **Development Environment**
 - ✓ Version Control
 - ✓ Python Environment
 - ✓ pip
 - ✓ virtualenv
 - ✓ Package Manager
 - ✓ Integrated Development Environment

Data Modelling

Data Modelling

- Data modelling is the process of defining and analysing data requirements of an information system.
- The process of data modelling involves:
 1. Identifying the detailed system requirements
 2. Normalising the entities
 3. Writing the data model
- We'll use UML to develop a formalised entity-relationship diagram (ERD), using Crow's foot notation.

Example Website: TIL

- We are going to consider a social ‘Today I learned’ platform – TIL.

The image displays three screenshots of the TIL (Today I Learned) website interface.

Home Screenshot: The top navigation bar includes links for Home, Create Post, and Log out. The main content area greets the user with "Hello, Josh" and identifies the site as "TIL: Your personal learning journal". It features a "Create a new post" button and a section titled "Your posts" showing a list of entries, such as "TIL why the HTTP header Referer isn't Referrer" and "TIL how to make a basic Django website". Below this is a "Tags you have used" section with tags like "django" and "python". At the bottom, there is a "Sign up" button.

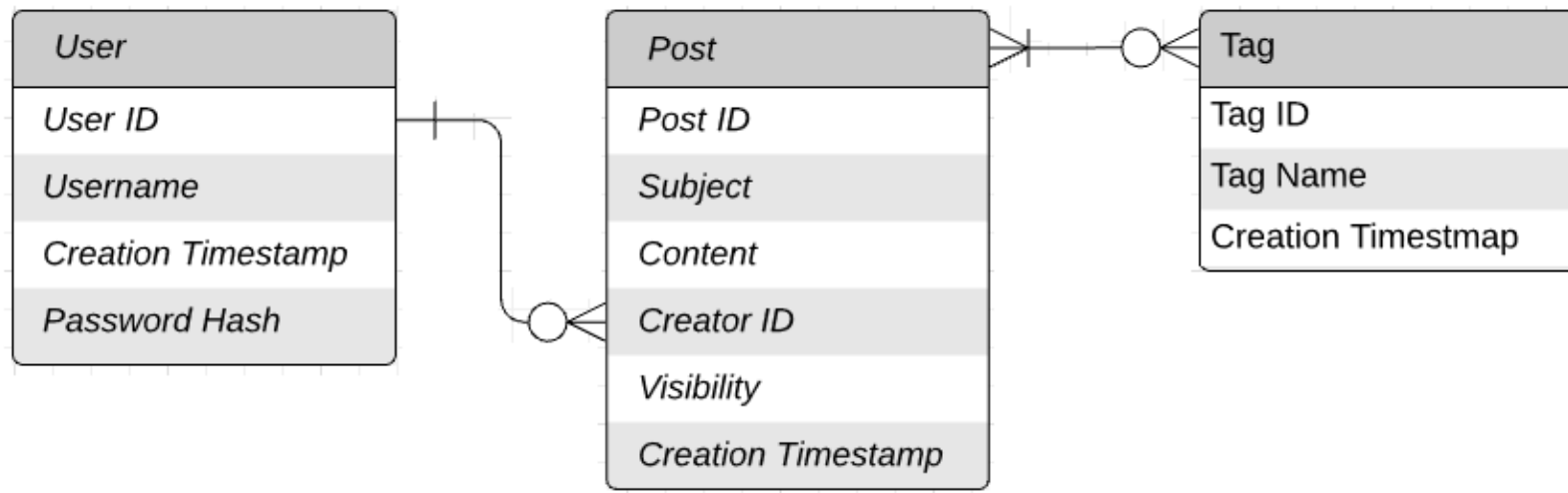
Login Screenshot: This view shows a large "Login" button. Below it are input fields for "Username:" and "Password:", each with a toggle for visibility. A "Login" button is positioned at the bottom of the form.

Create your post Screenshot: The top navigation bar remains the same. The main heading is "Create your post" with the subtitle "TIL: Your personal learning journal". The form includes a "Title" input field, a large text area for "What did I learn today?", and a "Tags (comma separated, maximum: 4)" input field. There is a checkbox for "Public" and a "Post" button with a star icon. Below the form, a section titled "Your recent posts" displays a list of recent entries, including "TIL why the HTTP header Referer isn't Referrer" and "TIL how to make a basic Django website".

TIL Rough Requirements

- A user can sign up with a username and password.
- Users can make **posts**, consisting of:
 - Subject
 - Content
 - Post date
- Posts can be:
 - public (visible to all users) or private (visible only to the creator)
 - associated with zero, one or more **tags**
- Tags are free-form text labels are dynamically created when first used.

Entity Relationship Diagram (ERD)

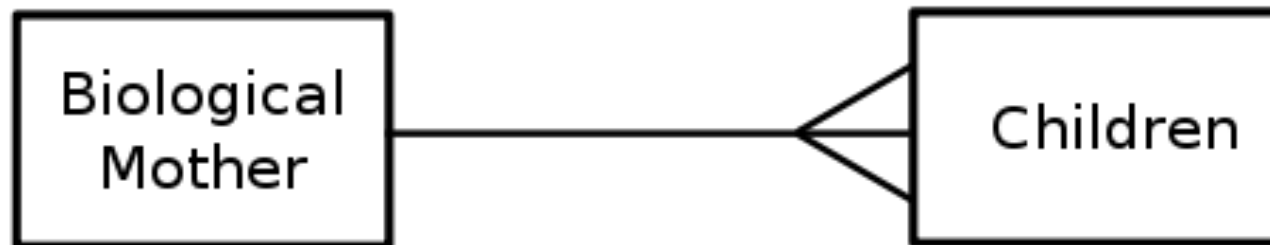


Models

- A model describes the fields which hold information about a particular entity. Each attribute of a model has a designated type, such as:
 - String
 - Numeric
 - Boolean
 - Date/Time
- An entity is represented by a single table in a database.
- Each table should have an 'id' field, which uniquely identifies each row, as the primary key.
- Each of the entities in an ERD can be considered a **model**.

Foreign Keys

- Models rarely exist in isolation, they have *relationships* with other models.
- A relationship to another model is expressed through a Foreign Keys (FK).
- Foreign Keys specify a ‘**one-to-many**’ relationship.



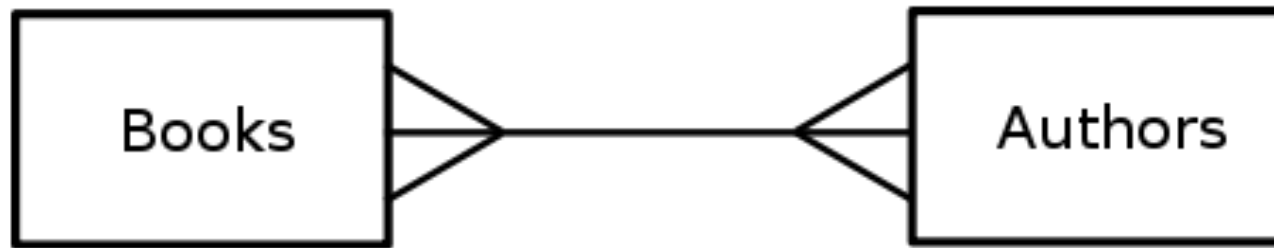
“**one-to-many**”: A mother may have many children, but a child has only one mother ([source](#))

TIL Example

A Post can be associated with one User.

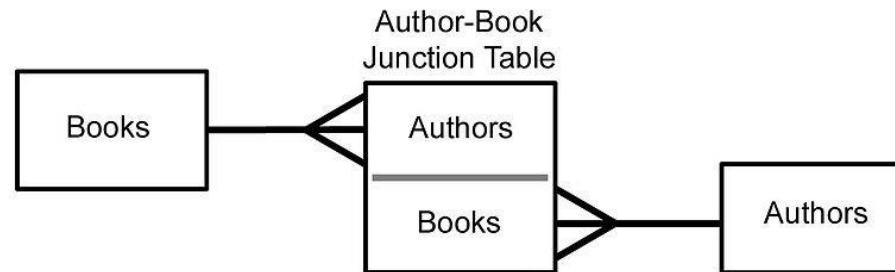
Posts contain a foreign key to their associated User.

Many-to-Many Relationships



“**many-to-many**”: An Author can write several Books, and a Book can be written by several Authors ([source](#))

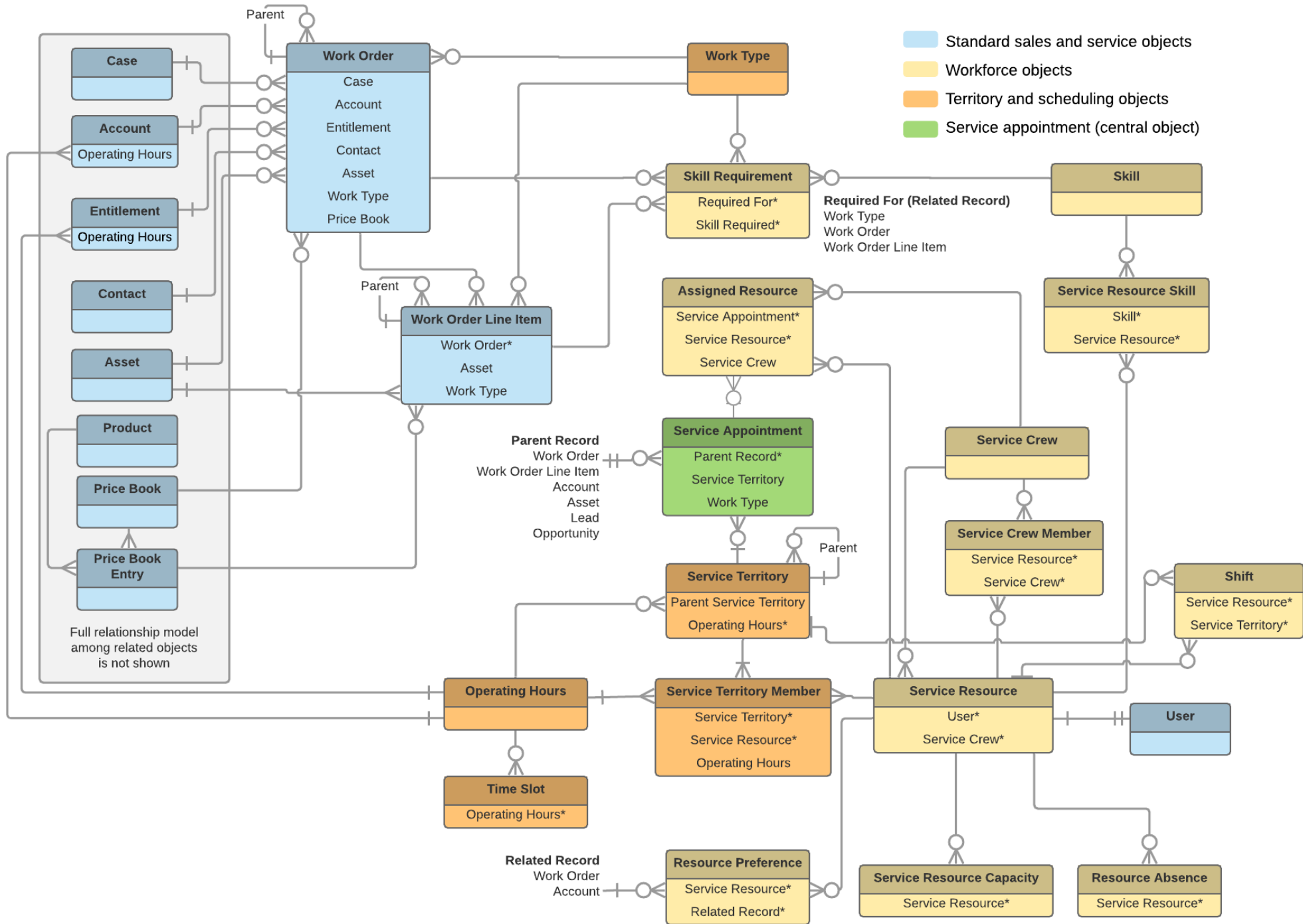
- To achieve a `**many-to-many**` relationship, we typically use intermediate tables to express the relationship.



TIL Example

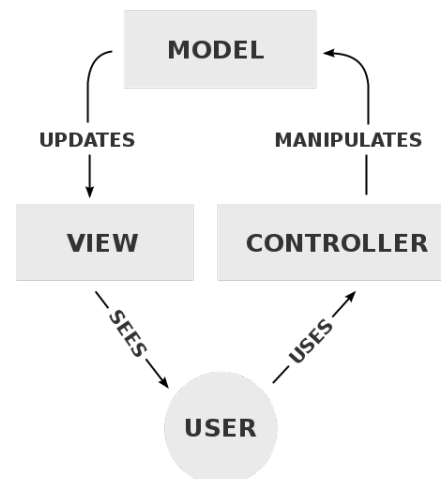
A *Post* can be associated with **many** *Tags*.

A *Tag* can be applied to **many** *Posts*



Model-View-Controller Architecture

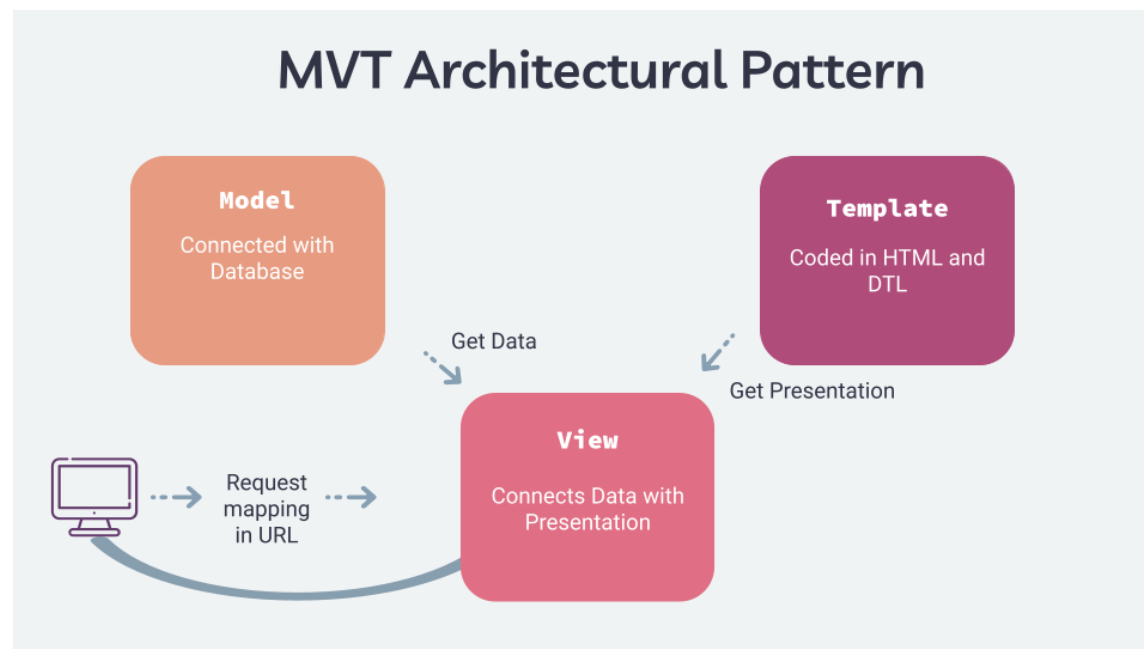
- A common way to structure the code of a web application is using the model-view-controller (MVC) architecture.
- In this pattern, code functionality is separated into three responsibilities:
 - **Model:** Handles the data.
 - **View:** Renders data into views (usually HTML).
 - **Controller:** Interprets requests and encapsulates business logic.



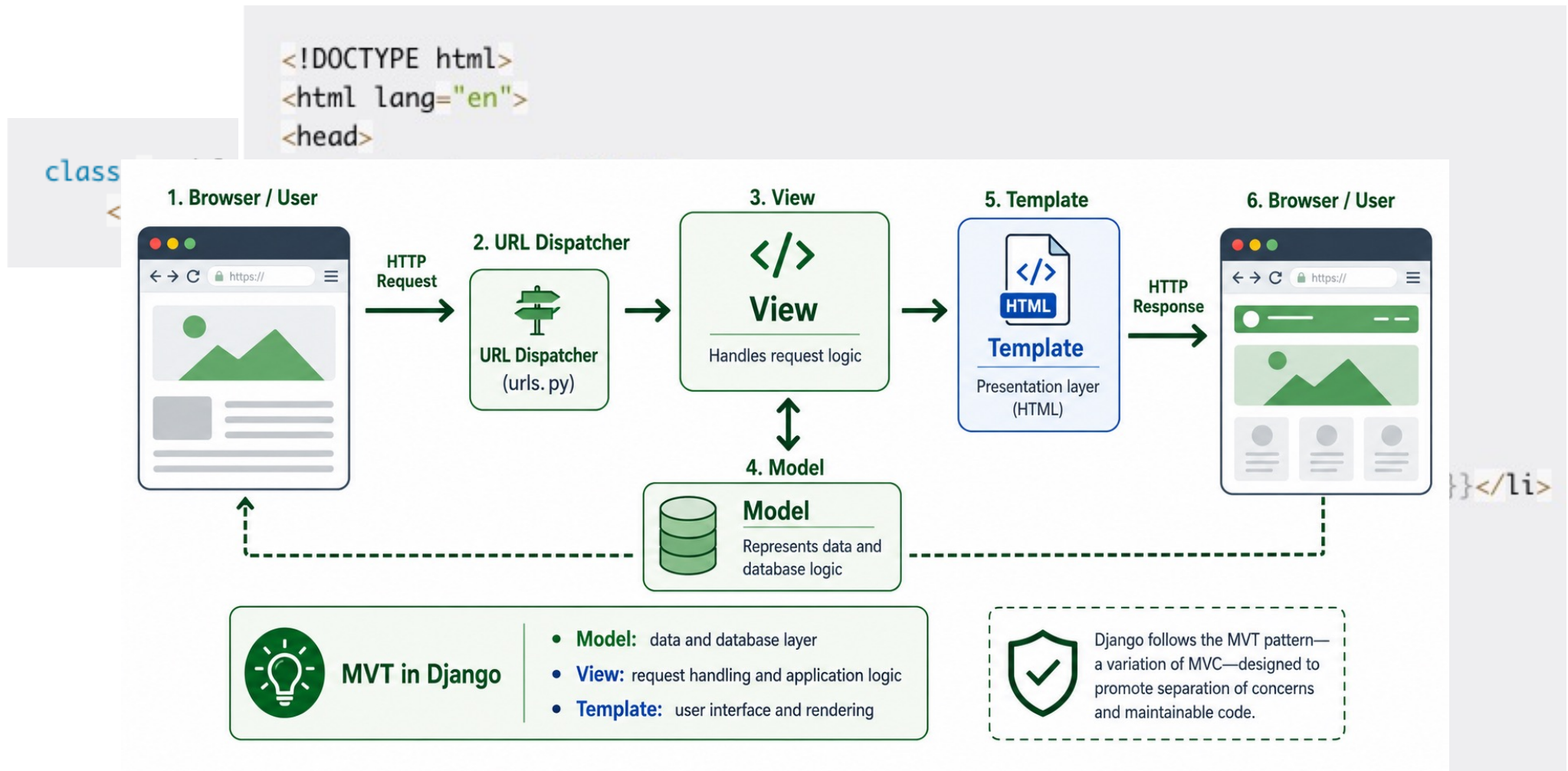
MVC Overview ([source](#))

Model-View-Template Architecture

- Model, similar to MVC handles the data and logic part, maintains a connection with the database.
- View works differently than the View-layer of MVC. Here, the view accepts the requests and sends out the response.
- Template, this is the presentation layer. It is commonly coded in HTML or DTL (Django Template Language).



Django



Development Environment

Version Control

- A version control system (VCS) helps to keep track of the changes to a file or files. It helps us to avoid this situation:
 - Report_draft.doc
 - Report_draft_final.doc
 - Report_draft_finalv2.doc
 - Report_draft_final - Copy.doc
 - Report_fixed.doc
 - Report_SUBMITTED.doc
- There are many other benefits:
 - Improved ability to collaborate
 - Compare changes made to a file over time
 - Revert to a previous version of a file

Common Version Control Software

- Common version control software:
 - Git: Industry standard tool, and what we will use in this course. Git integration is supported by GitHub and BitBucket, two leading online repository hosting providers.
 - SVN (Apache Subversion): Older VCS that is fading in popularity.
 - Hg (Mercurial): Alternative VCS written in Python.

Practise Git!

Git is used extensively in this course, and in your career. The sooner you master it, the easier your life will be.

- If you are not familiar with this tool, you should look at the following material:
 - Simple Git Guide
 - Github interactive introduction
 - Git Book - Basics

Python

- Python is the prescribed backend language for this course.
- It is an interpreted high-level programming language.
- Tutorials:
 - W3Schools Python Tutorial
 - [learnpython.org](https://www.learnpython.org/)
 - INFO1110: Intro to Programming
 - ELEC1005: Intro to Software Engineering

Familiarise yourself

Read the docs: [Python Documentation](https://docs.python.org/3/)
Installation: [Installation Guide](https://docs.python.org/3/using/windows.html)

Package Manager

- A package manager is a piece of software for managing dependencies. These can be libraries or extensions which add additional functionality or features to the language.
- Some common package managers:
 - Python: pip
 - Node: NPM
 - PHP: Composer
 - Ruby: RubyGems

Python Package Manager

- pip is the standard Python Package Manager.
- It allows us to easily install Python packages onto our system.

```
pip install django
```

- Will fetch and install the Django package and its dependencies.

Familiarise yourself

Read the docs: <https://pip.pypa.io/en/stable/>

Conflicting Project Dependencies

- Python only allows us to install one version of a package at a time.
- How do we handle conflicting project dependencies?
 - Awesome Website Requirements
 - `simplejson == 3.16`
 - `django == 3.1`
 - Better Website Requirements
 - `simplejson <= 3.17`
 - `django >= 3.2`
- Not possible to find a set of requirements that satisfies the desired versions.
 - Symbolic links?
 - Venv?



Virtual Environments

- To solve this problem, we can use Python Virtual Environments.
- Virtual environments allow us to have multiple, isolated installations of Python. Each virtual environment can have its own Python dependencies without conflicting with each other.
- A virtual environment must be activated before it can be used.
- Do not commit your virtual environment into your repository.

Virtual Environments in Python 3

- The venv module was shipped since Python 3.3, meaning support for virtual environments is built-in!
- Each venv has its **own site-packages** and Python executable, isolated from the system install
- To create a virtual environment, we run the command:

python3 -m venv myenvname

Virtual Environments

We will practice this during the tutorials

Integrated Development Environment

- An Integrated Development Environment (IDE) makes writing code easier by combining a text editor with additional features:
 - Debugger
 - Syntax highlighting
 - Version control
 - Refactoring
 - Search
- Some commonly used IDEs:
 - Visual Studio Code
 - PyCharm
 - Eclipse

**You reached the end of the lecture.
Thank you and see you next week.**